UNIVERSIDAD AUTÓNOMA DE CHIHUAHUA

INGENIERÍA EN CIENCIAS DE LA COMPUTACIÓN



CÓMPUTO PARALELO Y DISTRIBUIDO

Proyecto

ALUMNO: Joel Omar Castillo Castillo

Matricula: 367510

10 de Noviembre de 2026

Se utiliza la base de datos sample_ flix que ya viene con atlas su configuración y carga se encuentran en los archivos .env y common.py

```
MONGODB_URI=mongodb+srv://a367510_db_user:computo123@cluster0.nhng5cq.mongodb.net/?retryWrites=true&w=majority
DB_NAME=sample_mflix
COLLECTION_NAME=movies
```

el archivo env solo se encarga de jalar el link del cluster, la base de datos que se va a manejar y la colección donde estan las peliculas

Utilice el archivo common para no repetir el codigo, ya que ambos códigos, sincrono y asincrono hacen muchas cosas igual puse lo que ambas comparten como leer la configuración de el .env armar los filtros de búsqueda, generar la carga y dar formato a los datos cuando se crea o edita una película

el bloque mas importante o sobre el cual quiero hacer mas enfasis es donde se generan las películas(se hace la carga masiva) utilizando datos aleatorios

```python
#genrar las 1000 peliculas con datos
def make_movies(n=BULK_TOTAL):
    return [{
        "title": f"Pelicula Masiva {i}",
        "year": random.randint(1950, 2026),
        "genres": random.sample(GENRES, 2),
        "plot": "Pelicula de carga masiva",
        "runtime": random.randint(80, 180),
        "imdb": {"rating": round(random.uniform(3, 9.5), 1)},
        "bulk": True,
    } for i in range(1, n + 1)]
```

Y este el el bloque que se encarga de ayudar con el crud, elimina espacios en blanco de los titulos, transforma en año en un numero entero, jala el texto y convierte la calificacion en un float

```python
#json a formulario para mongo
def movie_from_body(b):
    return {
        "title": b["title"].strip(),
        "year": int(b["year"]),
        "genres": [g.strip() for g in b.get("genres", "").split(",") if g.strip()],
        "plot": b.get("plot", ""),
        "imdb": {"rating": float(b.get("rating") or 0)},
    }
```

Ahora pasamos las consultas, la carga y el CRUD de la forma síncrona y asíncrona

ASÍNCRONA
la primera función asíncrona que se usa es precisamente la que mantiene al servidor funcionando

```python
@asynccontextmanager #para apagar la conexion
async def lifespan(app):
    yield
    await client.close() #await es lo que lo hace asincrono
```

yield mantiene la conexión, al cerrar el servidor await ejecuta la orden de cerrar el servidor pero antes terminar los procesos que estan en marcha

después utilizamos otra función asíncrona al mostrar la pagina lo volvemos una función asíncrono apara que se vuelva no bloqueante y acepte múltiples cargas de diferentes usuarios en un caso hipotético. al definir como async mandamos esta función al event loop convirtiéndola en una corrutina

```python
@app.get("/", response_class=HTMLResponse)#esto muestra el index
async def home(request: Request):
    return templates.TemplateResponse(request, "index.html", {"mode": "ASÍNCRONO"})
```

la parte de consultas funciona de manera muy similar, primero dejamos el counter para medir el tiempo que se tarda, luego se filtra para buscar películas que tengan el poster. como definimos la función como asíncrona y pusimos un await slata a lo que sigue en lo que espera respuesta, esta funcion es asíncrona pero sigue una forma secuencial al llega a docs ya que si bien usan await para pasar a otras tareas en lo que esperan respuesta, el await de docs no se atendera hasta terminar lo que corresponde a cursor

```python
#CONSULTAS
@app.get("/api/random")  #genera las 5 peliculas
async def random_movies():
    t = time.perf_counter()
    cursor = await movies.aggregate([{"$match": {"poster": {"$exists": True}}}, {"$sample": {"size": 5}}])
    docs = await cursor.to_list(5)
    return {"movies": [clean(d) for d in docs], "ms": round((time.perf_counter() - t) * 1000)}


@app.get("/api/search")
async def search(q: str = "", genre: str = "", year_min: str = "", year_max: str = "", min_rating: str = ""):
    t = time.perf_counter()
    f = build_filter(q, genre, int(year_min or 0), int(year_max or 0), float(min_rating or 0))
    # asyncio.gather: la búsqueda y el conteo se ejecutan al mismo tiempo
    docs, total = await asyncio.gather(
        movies.find(f).sort("imdb.rating", -1).limit(20).to_list(20),
        movies.count_documents(f),
    )
    return {"movies": [clean(d) for d in docs], "total": total, "ms": round((time.perf_counter() - t) * 1000)}
```

en la parte de búsqueda utilizo **`asyncio.gather`** para lanzar dos corrutinas que comparten un mismo await, esto hace que su ejecución sea conjunta y no secuencial como se hacia al generar las películas de el inicio

```python
@app.get("/api/search")
async def search(q: str = "", genre: str = "", year_min: str = "", year_max: str = "", min_rating: str = ""):
    t = time.perf_counter()
    f = build_filter(q, genre, int(year_min or 0), int(year_max or 0), float(min_rating or 0))
    # asyncio.gather: la búsqueda y el conteo se ejecutan al mismo tiempo
    docs, total = await asyncio.gather(
        movies.find(f).sort("imdb.rating", -1).limit(20).to_list(20),
        movies.count_documents(f),
    )
    return {"movies": [clean(d) for d in docs], "total": total, "ms": round((time.perf_counter() - t) * 1000)}
```

SÍNCRONA

Aquí a diferencia del asíncrono no usamos FastAPI ni `AsyncMongoClient`, usamos Flask y el `MongoClient` normal de toda la vida, lo que significa que todo corre de forma completamente secuencial y bloqueante. No hay corrutinas, no hay Event Loop ni palabras clave `async` o `await`.

1. Conexión y mostrar la página

La conexión se abre directamente con `client = MongoClient(MONGODB_URI)`. Aquí no hay ningún `lifespan` ni `yield` administrando la vida del servidor; la conexión se queda amarrada al proceso de Flask.

Al entrar a la ruta principal con `@app.get("/")`, definimos una función normal `def home()`:

- No es una corrutina porque no lleva `async`.
- Simplemente lee el HTML de disco con `render_template` y se lo manda al usuario.
- En un caso hipotético con múltiples usuarios al mismo tiempo, esta función sí tiene que esperar su turno si el hilo del servidor está ocupado atendiendo otra petición pesada, porque corre de forma bloqueante.

2. Consultas de películas aleatorias (`/api/random`)

Al igual que en el asíncrono, primero iniciamos el `time.perf_counter()` para medir el tiempo. Luego hacemos el `movies.aggregate` para filtrar las películas que tengan póster y pedir 5 al azar con `$sample`.

- La gran diferencia con el asíncrono es que aquí no usamos `await cursor.to_list(5)`.
- Directamente envolvemos la consulta en `list(...)`. El hilo de Python se congela por completo esperando a que la base de datos termine de buscar y empaquetar las

5 películas. Mientras esa espera sucede, Python no puede saltar a ninguna otra tarea.
- Una vez que la lista `docs` se llena en memoria, se utiliza `jsonify` para transformar el diccionario en un JSON y mandarlo a la página junto con los milisegundos calculados.

3. Búsqueda y conteo (`/api/search`)

Aquí es donde se nota el mayor contraste con el `asyncio.gather` del archivo asíncrono:

- En lugar de lanzar las dos cosas al mismo tiempo compartiendo un `await`, aquí se ejecutan forzosamente una detrás de otra en fila india.
- Primero se ejecuta la línea de `docs = list(movies.find(f)...)`. Python le pide a MongoDB las 20 películas ordenadas por calificación y se queda esperando con los brazos cruzados a que la base de datos responda.
- Hasta que esa búsqueda termina y llegan los datos, apenas pasa a la siguiente línea: `total = movies.count_documents(f)`. Vuelve a congelarse esperando a que la base de datos cuente cuántas películas cumplieron el filtro en toda la colección.
- Como los tiempos de ambas consultas se suman uno tras otro en lugar de hacerse en paralelo, esta función tarda significativamente más tiempo que en el asíncrono.

4. Carga masiva (`/api/bulk`)

Para cargar las 1,000 películas inventadas de `common.py`:

- En el asíncrono teníamos a `TaskGroup` aventando los 10 lotes juntos y al semáforo dejando pasar de 3 en 3.
- Aquí no hay semáforo ni grupos de tareas: usamos un simple ciclo `for` tradicional.
- El ciclo `for` agarra el primer lote de 100 películas (`batch`), ejecuta el `movies.bulk_write` y bloquea la ejecución hasta que ese lote se escribe en la base de datos.
- Solo cuando ese lote terminó, el ciclo da la siguiente vuelta, manda el segundo lote de 100 y vuelve a esperar.
- Se hacen 10 viajes individuales y consecutivos. No hay solapamiento ni paralelismo de ningún tipo: viaja el lote 1, termina, viaja el 2, termina, y así hasta llegar a las 1,000 películas.

5. El CRUD (POST, PUT, DELETE en `/api/movies`)

Funciona exactamente igual que en el asíncrono respecto a la lógica:

- Para crear una película usa `movies.insert_one` pasándolo por `movie_from_body`.
- Para editar usa `movies.update_one` con el `ObjectId` y `$set`.
- Para eliminar usa `movies.delete_one`.

- La única diferencia técnica: ninguna de estas operaciones lleva `await`. Cada vez que guardas, editas o borras una película, la función bloquea el hilo de ejecución hasta que la base de datos confirma el cambio, antes de poder devolver el `jsonify` al navegador.